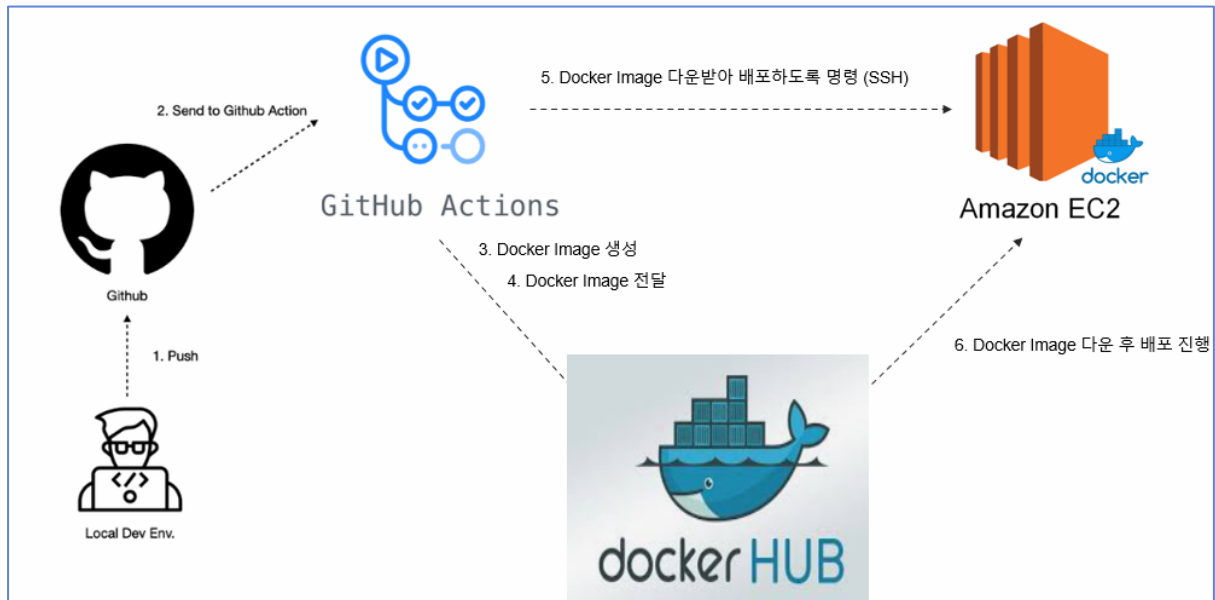


컨테이너 기반(Docker)의 CI/CD

Github Actions에서 빌드해서 배포할 예정이므로 프로덕션 서버의 성능에 영향을 주지 않는다.

전체흐름



✓ GitHub Actions를 통한 Docker 기반 배포 전체 흐름

- 개발자가 로컬 환경에서 소스 코드를 작성하고 GitHub에 Push**
→ 변경된 코드가 GitHub 저장소에 업로드됨.
- GitHub 저장소에 Push 되면 GitHub Actions 트리거 발생**
→ .github/workflows/*.yml에 정의된 CI/CD 파이프라인 실행.
- GitHub Actions가 Docker Image 생성**
→ Dockerfile을 기준으로 애플리케이션을 컨테이너 이미지로 빌드.
- 생성된 Docker Image를 Docker Hub에 푸시(Push)**
→ GitHub Actions가 docker login → docker push 명령을 통해 이미지를 Docker Hub에 업로드.
- GitHub Actions가 EC2에 SSH로 접속하여 Docker Image 다운로드 명령 실행**
→ EC2에 docker pull <image> 명령을 내려 배포 준비.
- EC2가 Docker Hub에서 이미지 다운로드 후 컨테이너로 실행하여 배포 완료**
→ docker run 또는 docker-compose up 등을 통해 컨테이너로 실행.

실습하기(제공파일 > ex05_Docker기반 폴더)

기존에 사용한 ec2를 그대로 사용 할 예정이므로 ec2에 기존 프로젝트 폴더를 삭제한다.

8080번 포트에 실행되고 있는 프로세스 종료

```
sudo fuser -k -n tcp 8080
```

8080번 포트에 Spring Boot가 실행되고 있는 지 확인

```
sudo lsof -i:8080
```

폴더 삭제

```
sudo rm -rf cicd-server
```

브라우저로 중지된 상황 확인



Ubuntu에서 Docker, Docker Compose 설치하기

```
sudo apt-get update && ₩
sudo apt-get install -y apt-transport-https ca-certificates curl software-properties-common && ₩
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add - && ₩
sudo apt-key fingerprint 0EBFCD88 && ₩
sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"
&& ₩
sudo apt-get update && ₩
sudo apt-get install -y docker-ce && ₩
sudo usermod -aG docker ubuntu && ₩
sudo curl -L "https://github.com/docker/compose/releases/download/1.23.2/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose && ₩
sudo chmod +x /usr/local/bin/docker-compose && ₩
sudo ln -s /usr/local/bin/docker-compose /usr/bin/docker-compose
```

Docker 버전 확인

```
docker -v
```

Docker Compose 버전 확인

```
docker compose version
```

프로젝트 안에 Dockerfile 작성

```
# 1. OpenJDK 17 기반 이미지 사용
FROM eclipse-temurin:17-jdk

# 2. JAR 파일 복사
COPY build/libs/*SNAPSHOT.jar app.jar

# 3. 포트 오픈
EXPOSE 8080

# 4. JAR 실행
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

.github/workflows/deploy.yml 내용

```
1  # GitHub Actions 워크플로우 이름
2  name: Docker 컨테이너로 EC2에 배포하기
3
4  # main 브랜치에 push가 발생하면 이 워크플로우 실행
5  on:
6    push:
7      branches:
8        - main
9
10 jobs:
11   deploy:
12     runs-on: ubuntu-latest # GitHub 제공 Ubuntu 환경에서 실행
13
14     steps:
15       # 1. GitHub 저장소 코드 가져오기
16       - name: Checkout source
17         uses: actions/checkout@v4
18
19       # 2. Spring Boot 설정 파일(application.properties)을 생성
20       # GitHub Secrets에 저장된 문자열을 파일로 생성
21       - name: Create application.properties
22         run: echo "${{ secrets.APPLICATION_PROPERTIES }}" > ./src/main/resources/application.properties
23
24       # 3. docker-compose.yml 파일 생성
25       # GitHub Secrets에 저장된 내용 기반으로 로컬에 생성
26       - name: Create docker-compose.yml 생성
27         run: echo "${{ secrets.DOCKER_COMPOSE }}" > ./docker-compose.yml
28
29       # 4. gradlew에 실행 권한 부여 (리눅스에서 실행되도록)
30       - name: Gradlew 실행 권한 부여
31         run: chmod +x ./gradlew
32
33       # 5. 프로젝트를 빌드 (클리 후 재빌드)
34       - name: Gradle 빌드 실행
35       run: ./gradlew clean build
```

```

37 # 6. Docker Hub에 로그인 (푸시하기 위해 인증)
38 - name: Docker Hub 로그인
39   uses: docker/login-action@v3
40   with:
41     username: ${ secrets.DOCKER_USERNAME }
42     password: ${ secrets.DOCKER_PASSWORD }
43
44 # 7. 빌드된 JAR 파일을 포함한 Docker 이미지 생성
45 - name: Docker 이미지 빌드
46   run: docker build -t ${ secrets.DOCKER_USERNAME }/myapp:latest .
47
48 # 8. Docker Hub에 이미지 푸시 (EC2에서 사용하기 위해)
49 - name: Docker 이미지 푸시
50   run: docker push ${ secrets.DOCKER_USERNAME }/myapp:latest
51
52 # 9. docker-compose.yml 파일을 EC2 서버로 복사
53 - name: docker-compose.yml 전송
54   uses: appleboy/scp-action@v0.1.7
55   with:
56     host: ${ secrets.EC2_HOST } # EC2 IP
57     username: ${ secrets.EC2_USERNAME } # EC2 사용자명 (보통 ubuntu)
58     key: ${ secrets.EC2_PRIVATE_KEY } # EC2 접근용 개인키
59     source: "docker-compose.yml" # 로컬에서 보낼 파일
60     target: "~/cid-docker/" # EC2 내 저장 위치
61
62 # 10. EC2에 접속하여 docker-compose 명령 실행
63 - name: EC2에서 컨테이너 실행
64   uses: appleboy/ssh-action@v1.0.3
65   with:
66     host: ${ secrets.EC2_HOST }
67     username: ${ secrets.EC2_USERNAME }
68     key: ${ secrets.EC2_PRIVATE_KEY }
69     script_stop: true
70     script: |
71       mkdir -p ~/cid-docker # 배포 디렉토리 생성 (이미 있으면 무시)
72
73       cd ~/cid-docker
74       docker-compose down || true # 기존 컨테이너 중지 (실패해도 계속)
75       docker-compose pull # 최신 이미지 가져오기
76       docker-compose up -d # 백그라운드에서 컨테이너 실행
77

```

.github/workflows/deploy.yml 수정

GitHub Actions 워크플로우 이름

name: Docker 컨테이너로 EC2에 배포하기

main 브랜치에 push가 발생하면 이 워크플로우 실행

on:

push:

branches:

```
- main

jobs:
  deploy:
    runs-on: ubuntu-latest # GitHub 제공 Ubuntu 환경에서 실행

    steps:
      # 1. GitHub 저장소 코드 가져오기
      - name: Checkout source
        uses: actions/checkout@v4

      # 2. Spring Boot 설정 파일(application.properties)을 생성
      # GitHub Secrets에 저장된 문자열을 파일로 생성
      - name: Create application.properties
        run: echo "${{ secrets.APPLICATION_PROPERTIES }}" > ./src/main/resources/application.properties

      # 3. docker-compose.yml 파일 생성
      # GitHub Secrets에 저장된 내용 기반으로 로컬에 생성
      - name: Create docker-compose.yml 생성
        run: echo "${{ secrets.DOCKER_COMPOSE }}" > ./docker-compose.yml

      # 4. gradlew에 실행 권한 부여 (리눅스에서 실행되도록)
      - name: Gradlew 실행 권한 부여
        run: chmod +x ./gradlew

      # 5. 프로젝트를 빌드 (클린 후 재빌드)
      - name: Gradle 빌드 실행
        run: ./gradlew clean build

      # 6. Docker Hub에 로그인 (푸시하기 위해 인증)
      - name: Docker Hub 로그인
        uses: docker/login-action@v3
        with:
          username: "${{ secrets.DOCKER_USERNAME }}"
          password: "${{ secrets.DOCKER_PASSWORD }}"

      # 7. 빌드된 JAR 파일을 포함한 Docker 이미지 생성
      - name: Docker 이미지 빌드
        run: docker build -t "${{ secrets.DOCKER_USERNAME }}/myapp:latest" .

      # 8. Docker Hub에 이미지 푸시 (EC2에서 사용하기 위해)
      - name: Docker 이미지 푸시
        run: docker push "${{ secrets.DOCKER_USERNAME }}/myapp:latest"

      # 9. docker-compose.yml 파일을 EC2 서버로 복사
      - name: docker-compose.yml 전송
        uses: appleboy/scp-action@v0.1.7
        with:
          host: "${{ secrets.EC2_HOST }}" # EC2 IP
```

```

username: ${ secrets.EC2_USERNAME }} # EC2 사용자명 (보통 ubuntu)
key: ${ secrets.EC2_PRIVATE_KEY }} # EC2 접근용 개인키
source: "docker-compose.yml" # 로컬에서 보낼 파일
target: "~/cicd-docker/" # EC2 내 저장 위치

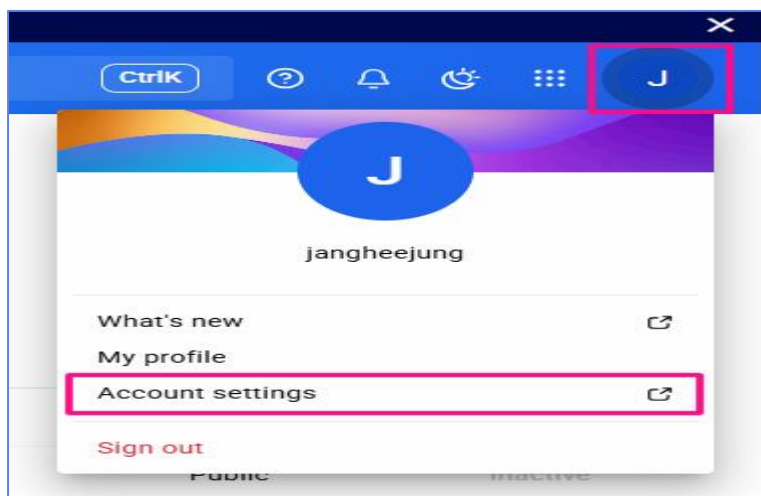
# 10. EC2에 접속하여 docker-compose 명령 실행
- name: EC2에서 컨테이너 실행
  uses: appleboy/ssh-action@v1.0.3
  with:
    host: ${ secrets.EC2_HOST }}
    username: ${ secrets.EC2_USERNAME }}
    key: ${ secrets.EC2_PRIVATE_KEY }}
    script_stop: true
    script: |
      mkdir -p ~/cicd-docker # 배포 디렉토리 생성 (이미 있으면 무시)

      cd ~/cicd-docker
      docker compose down || true # 기존 컨테이너 중지 (실패해도 계속)
      docker compose pull # 최신 이미지 가져오기
      docker compose up -d # 백그라운드에서 컨테이너 실행

```

Docker Hub Access Token을 새로 생성 하기

Docker Hub Access Token은 Docker Hub에 로그인할 때 사용하는 비밀번호 대신 사용할 수 있는 안전한 인증 수단이다. 비밀번호보다 더 안전하게 인증을 처리할 수 있게 해 주며, 특히 자동화된 스크립트나 CI/CD 파이프라인에서 Docker Hub에 접근할 때 주로 사용한다.



<https://app.docker.com/settings/account-information>

Personal access tokens / New access token

Create access token

A personal access token is similar to a password except you can have many tokens and revoke access to each one at any time. [Learn more](#)

Access token description
cicd_token

Expiration date
None

Optional
Access permissions
Read & Write

Read & Write tokens allow you to push images to any repository managed by your account.

Cancel Generate

Copy access token

Use this token as a password when you sign in from the Docker CLI client. [Learn more](#)

Make sure you copy your personal access token now. Your personal access token is only displayed once. It isn't stored and can't be retrieved later.

Access token description
cicd_token

Expires on
Never

Access permissions
Read & Write

To use the access token from your Docker CLI client:

1. Run

```
$ docker login -u jangheejung
```

Copy

2. At the password prompt, enter the personal access token.

```
dckr_pat_CQXg5oozX3scskiZX1YpEL--eK0
```

Copy

→ copy해서 다른 메모장에 저장해 놓는다.

[Back to access tokens](#)

GitHub Secrets에 추가 등록

이름	설명
DOCKER_USERNAME	Docker Hub ID
DOCKER_PASSWORD	Docker Hub Access Token
DOCKER_COMPOSE	docker-compose.yml 파일 내용 전체 (멀티라인 YAML 문자열)

Actions secrets / New secret

Name *

DOCKER_USERNAME

Secret *

jangheejung

Actions secrets / New secret

Name *

DOCKER_PASSWORD

Secret *

docker hub accss token 넣는다.

Actions secrets / New secret

Name *

DOCKER_COMPOSE

Secret *

version: '3.8'

services:

app:

image: jangheejung/myapp:latest

container_name: myapp

ports:

- "8080:8080"

restart: always

본인docker hub계정맞이름

Add secret

Repository secrets

Name ↕↑	Last updated
APPLICATION_PROPERTIES	4 hours ago
DOCKER_COMPOSE	5 minutes ago
DOCKER_PASSWORD	20 minutes ago
DOCKER_USERNAME	20 minutes ago
EC2_HOST	yesterday
EC2_PRIVATE_KEY	yesterday
EC2_USERNAME	yesterday

docker-compose.yml 내용

```
services:
  app:
    image: jangheejung/myapp:latest
    container_name: myapp
    ports:
      - "8080:8080"
    restart: always
```

.gitignore 수정

```
38
39     # application.properties or application.yml 무시
40     application.*
41     docker-compose.yml
42
```

작업 pc변경 사항 commit > push

```
git add .

# git 캐시 삭제
git rm -r --cached .

# 변경내용 statging에 올리기
git add .

# commit하기
git commit -m "docker로 배포하기 test"

# 원격에 push하기
git push origin main
```

Github Action workflow 확인

✓ 최종수정! #20

Summary

Jobs

✓ deploy

Run details

Usage

Workflow file

deploy

succeeded 1 minute ago in 1m 21s

- Set up job
- Build appleboy/scp-action@v0.1.7
- Build appleboy/ssh-action@v1.0.3
- Checkout source
- Create application.properties
- Create docker-compose.yml 생성
- Gradlew 실행 권한 부여
- Gradle 빌드 실행
- Docker Hub 로그인
- Docker 이미지 빌드
- Docker 이미지 푸시
- docker-compose.yml 전송
- EC2에서 컨테이너 실행
- Post Docker Hub 로그인
- Post Checkout source
- Complete job

브라우저에서 확인

← → ↻ ⚠ 주의 요함 3.36.53.40:8080

Chrome이 기본 브라우저로 설정되어 있지 않습니다 기본값으로 설정

Docker 기반의 GitHub Actions를 이용한 CI/CD 수정완료!!!

EC2에서 확인

```

ubuntu@ip-172-31-11-11:~$ ls
cicd-docker
ubuntu@ip-172-31-11-11:~$ cd cicd-docker/
ubuntu@ip-172-31-11-11:~/cicd-docker$ ls
docker-compose.yml
ubuntu@ip-172-31-11-11:~/cicd-docker$ sudo docker ps -a
CONTAINER ID        IMAGE                                     COMMAND                  CREATED             STATUS              PORTS
0b8e9126ba72       jangheejung/myapp:latest               "java -jar /app.jar"    2 minutes ago       Up 2 minutes       0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
ubuntu@ip-172-31-11-11:~/cicd-docker$ sudo docker images
REPOSITORY          TAG             IMAGE ID            CREATED             SIZE
jangheejung/myapp   latest         b85ffb99d8b7       3 minutes ago      440MB
  
```